



Python: Estructuras, Funciones y POO

Unidad 4.2 — Una guía completa para dominar las estructuras de datos, la programación funcional y la orientación a objetos en Python. Ideal para desarrolladores con experiencia en Java que desean profundizar en el ecosistema Python.

UNIDAD 4.2

NIVEL INTERMEDIO

¿Qué veremos en esta unidad?

Esta unidad cubre los pilares fundamentales de Python moderno: desde las estructuras de datos nativas hasta la programación orientada a objetos, pasando por técnicas funcionales y buenas prácticas profesionales.



Estructuras de datos

Listas, tuplas, diccionarios, conjuntos y comprensiones



Funciones avanzadas

Parámetros, lambda, *args, **kwargs, módulos y paquetes



Entrada/Salida y errores

Ficheros CSV y JSON, manejo de excepciones



POO y buenas prácticas

Clases, objetos, comparativa con Java, PEP 8 y docstrings



Listas y Tuplas

Las **listas** y las **tuplas** son las colecciones secuenciales más fundamentales de Python. Ambas almacenan elementos ordenados, pero difieren en mutabilidad: las listas son modificables, las tuplas son inmutables.

Listas: colecciones dinámicas

Características clave

- Ordenadas e indexadas desde 0
- Mutables: se pueden añadir, modificar y eliminar elementos
- Permiten elementos duplicados y tipos mixtos
- Operaciones: `append()`, `extend()`, `pop()`, `sort()`

Ejemplo práctico

```
frutas = ["manzana", "pera", "uva"]
frutas.append("mango")
frutas[1] = "melocotón"
frutas.sort()
print(frutas)
# ['mango', 'manzana', 'melocotón', 'uva']

# Slicing
primeras = frutas[:2]
# ['mango', 'manzana']
```

Tuplas: colecciones inmutables

Las tuplas se definen con paréntesis y, una vez creadas, **no pueden modificarse**. Son más eficientes en memoria y se usan para datos que no deben cambiar, como coordenadas, configuraciones o claves de diccionarios.

Cuándo usar tuplas

- Coordenadas geográficas: (lat, lon)
- Retorno múltiple de funciones
- Claves compuestas en diccionarios
- Datos de configuración constantes

Ejemplo

```
punto = (3.5, -7.2)
x, y = punto # desempaquetado

# Tupla como clave de diccionario
mapa = {(0, 0): "origen", (1, 0): "este"}

# Retorno múltiple
def minmax(lst):
    return min(lst), max(lst)

lo, hi = minmax([4, 1, 9, 2])
```


Diccionarios y Conjuntos

Los **diccionarios** almacenan pares clave-valor y permiten búsquedas en tiempo $O(1)$. Los **conjuntos** son colecciones no ordenadas de elementos únicos, ideales para eliminar duplicados y realizar operaciones de teoría de conjuntos.



Diccionarios: mapas clave-valor

Operaciones esenciales

```
alumno = {  
    "nombre": "Ana",  
    "nota": 9.5,  
    "módulos": ["Python", "SQL"]  
}  
  
# Acceso seguro  
print(alumno.get("edad", "N/D"))  
  
# Iterar  
for clave, valor in alumno.items():  
    print(f"{clave}: {valor}")  
  
# Dict comprehension  
cuadrados = {n: n**2 for n in range(5)}
```

Métodos útiles

- `keys()` — vista de claves
- `values()` — vista de valores
- `items()` — pares (clave, valor)
- `get(k, default)` — acceso seguro
- `update(otro_dict)` — fusionar
- `pop(k)` — eliminar y obtener

📌 Desde Python 3.7 los diccionarios mantienen el orden de inserción.

Conjuntos: colecciones únicas

Creación y operaciones

```
a = {1, 2, 3, 4}
```

```
b = {3, 4, 5, 6}
```

```
union      = a | b  # {1,2,3,4,5,6}
```

```
interseccion = a & b  # {3,4}
```

```
diferencia = a - b  # {1,2}
```

```
simetrica   = a ^ b  # {1,2,5,6}
```

```
# Eliminar duplicados de una lista
```

```
lista = [1, 2, 2, 3, 3, 4]
```

```
sin_duplicados = list(set(lista))
```

¿Cuándo usar conjuntos?

- Eliminar duplicados rápidamente
- Comprobar pertenencia en $O(1)$
- Operaciones de intersección y diferencia
- Filtrar elementos comunes entre colecciones

Los conjuntos no admiten indexación y sus elementos deben ser **hasheables** (inmutables).

Comprensiones: Python idiomático

Las **comprensiones** son una de las características más elegantes de Python. Permiten crear colecciones de forma concisa y legible, sustituyendo bucles `for` explícitos por una sintaxis compacta y expresiva.

Lista

```
[expr for x in it if cond]
```

Pares de una lista numérica:

```
[x for x in range(10) if x % 2  
== 0]
```

Diccionario

```
{k: v for k, v in items}
```

Invertir claves y valores: {v:

```
k for k, v in d.items()}
```

Conjunto

```
{expr for x in it}
```

Letras únicas: {c.lower() **for**

```
c in texto if c.isalpha()}
```

Generador

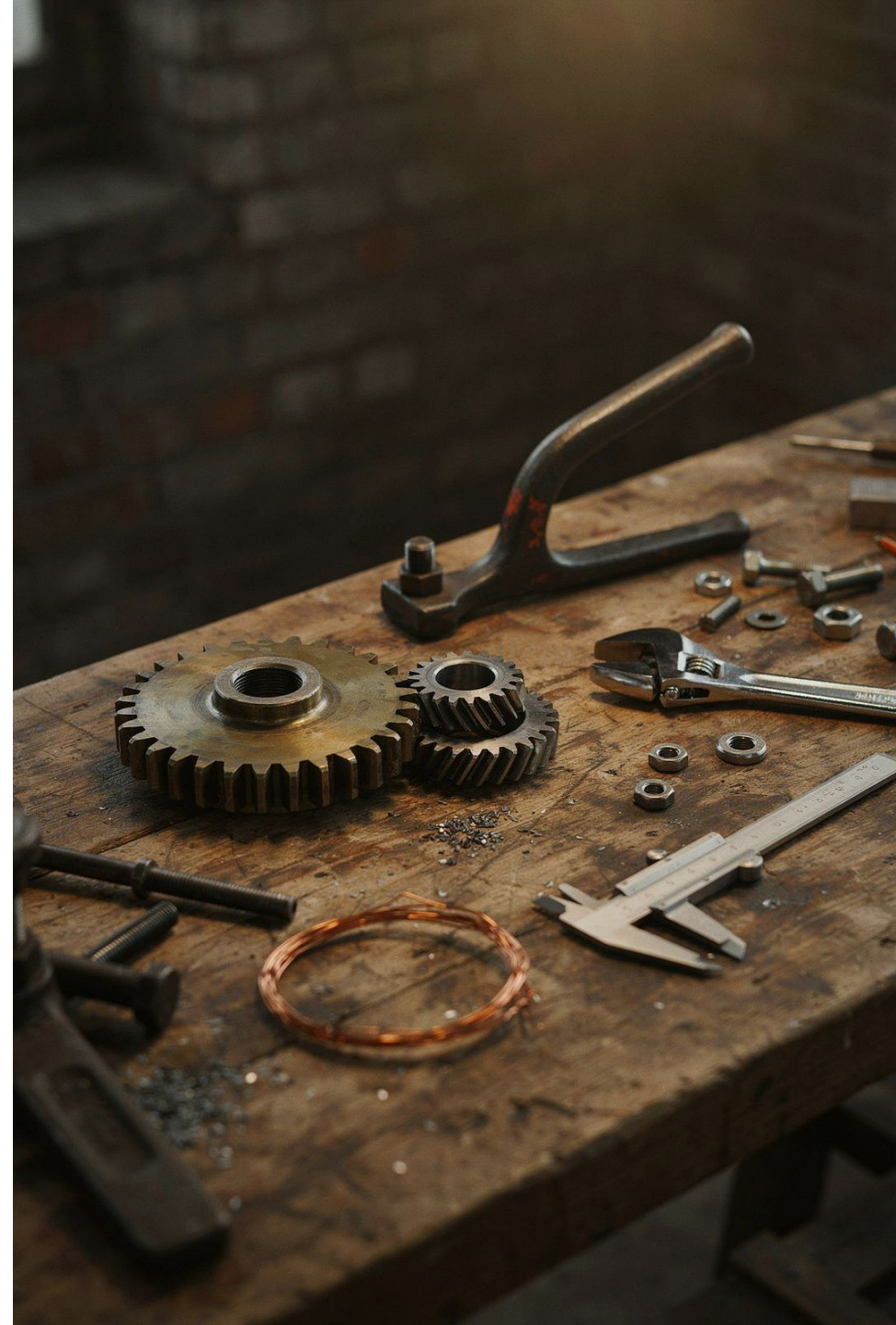
```
(expr for x in it)
```

Eficiente en memoria:

```
sum(x**2 for x in  
range(1000))
```

Funciones Avanzadas

Python ofrece mecanismos muy flexibles para definir funciones: parámetros por defecto, argumentos posicionales variables (`*args`), argumentos de palabra clave variables (`**kwargs`) y funciones anónimas con `lambda`.



Parámetros y funciones lambda

Tipos de parámetros

```
def conectar(host, puerto=5432,
             *args, **kwargs):
    print(host, puerto)
    print(args) # tupla extra
    print(kwargs) # dict extra

conectar("localhost", usuario="ana",
        ssl=True)
```

- **Posicionales:** obligatorios, en orden
- **Por defecto:** valor predefinido
- ***args:** número variable de posicionales
- ****kwargs:** número variable de nombrados

Funciones lambda

Las **lambdas** son funciones anónimas de una sola expresión. Útiles como argumentos de funciones de orden superior.

```
doble = lambda x: x * 2
suma = lambda a, b: a + b

# Uso con sorted y map
alumnos = [("Ana", 9), ("Bea", 7)]
alumnos.sort(key=lambda a: a[1],
             reverse=True)

pares = list(map(lambda x: x*2,
                 range(5)))
```

Módulos y Paquetes

Python organiza el código en **módulos** (archivos `.py`) y **paquetes** (directorios con `__init__.py`). Esta modularidad facilita la reutilización, el mantenimiento y la distribución del código.



Módulos estándar

Python incluye una biblioteca estándar enorme: `os`, `sys`, `math`, `datetime`, `random`, `re`... Importa solo lo que necesites con `from módulo import nombre`.



Paquetes propios

Agrupar módulos relacionados en un directorio con `__init__.py`. El archivo `__init__.py` puede estar vacío o exponer la API pública del paquete.



PyPI y pip

Instalar paquetes de terceros con `pip install nombre`. Usar `requirements.txt` o `pyproject.toml` para declarar dependencias de tu proyecto.

Ficheros: CSV y JSON

Trabajar con ficheros externos es imprescindible en cualquier aplicación real. Python proporciona módulos estándar para los dos formatos de intercambio de datos más habituales.

CSV con el módulo `csv`

```
import csv

# Lectura
with open("datos.csv", newline="",
          encoding="utf-8") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila["nombre"])

# Escritura
with open("salida.csv", "w",
          newline="") as f:
    campos = ["nombre", "nota"]
    escritor = csv.DictWriter(f,
                              fieldnames=campos)
    escritor.writeheader()
    escritor.writerow(
        {"nombre": "Ana", "nota": 9})
```

JSON con el módulo `json`

```
import json

datos = {"nombre": "Ana", "nota": 9.5}

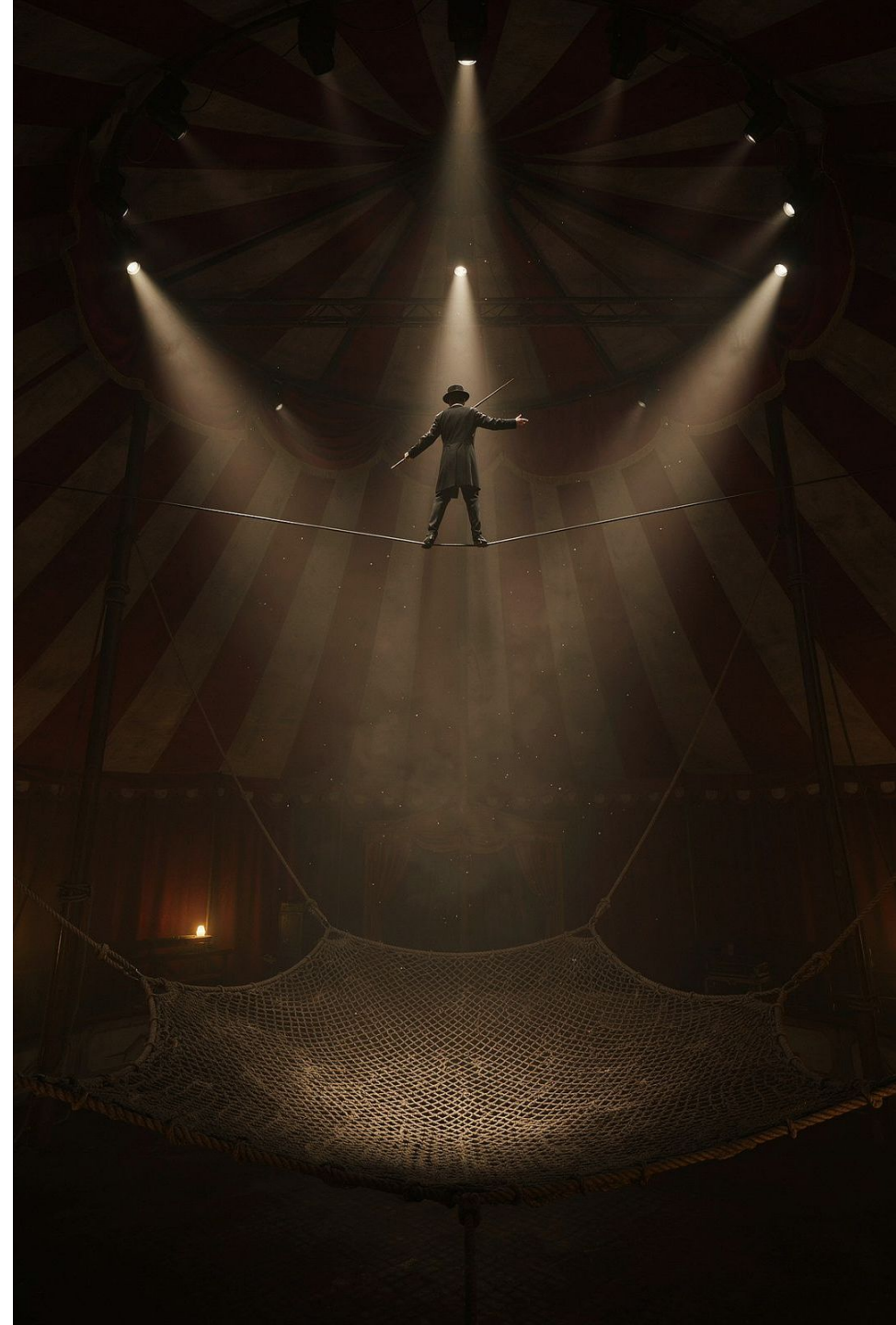
# Serializar a fichero
with open("datos.json", "w") as f:
    json.dump(datos, f, indent=2)

# Deserializar desde fichero
with open("datos.json") as f:
    cargado = json.load(f)

# Serializar a cadena
cadena = json.dumps(datos)
objeto = json.loads(cadena)
```


Manejo de Excepciones

Las excepciones permiten manejar errores en tiempo de ejecución de forma controlada, evitando que el programa termine abruptamente. Python usa el bloque `try / except / else / finally` para gestionar errores de manera estructurada.



Excepciones: estructura y jerarquía

Sintaxis completa

```
try:
    resultado = 10 / int(input("Divisor: "))
except ZeroDivisionError:
    print("No se puede dividir por cero")
except ValueError as e:
    print(f"Valor no válido: {e}")
except (TypeError, OverflowError):
    print("Error de tipo u overflow")
else:
    print(f"Resultado: {resultado}")
finally:
    print("Bloque siempre ejecutado")

# Lanzar excepciones propias
class NotaInvalidaError(ValueError):
    pass

def validar_nota(n):
    if not 0 <= n <= 10:
        raise NotaInvalidaError(
            f"Nota {n} fuera de rango")
```

Buenas prácticas

- Captura excepciones **específicas**, nunca `except Exception` genérico salvo en el nivel más alto
- Usa `else` para el código que solo se ejecuta si no hubo error
- Usa `finally` para liberar recursos (ficheros, conexiones)
- Crea excepciones personalizadas heredando de las clases estándar
- Considera el gestor de contexto `with` para recursos

📌 El bloque `else` es exclusivo de Python y no tiene equivalente directo en Java.

Clases y Objetos en Python

Python es un lenguaje orientado a objetos de primera clase. Sus clases son **dinámicas**, más flexibles que en Java, y soportan herencia múltiple, métodos especiales (*dunder methods*) y propiedades con decoradores.



POO: Python vs. Java

Python

```
class Estudiante:
    """Representa un estudiante."""

    num_estudiantes = 0 # atributo de clase

    def __init__(self, nombre, nota):
        self.nombre = nombre # público
        self._nota = nota # convención: protegido

    @property
    def nota(self):
        return self._nota

    @nota.setter
    def nota(self, valor):
        if 0 <= valor <= 10:
            self._nota = valor

    def __str__(self):
        return f"{self.nombre}: {self._nota}"

    def __repr__(self):
        return f"Estudiante({self.nombre!r})"
```

Diferencias clave con Java

Sin tipos estáticos

Python usa tipado dinámico. Las anotaciones de tipo son opcionales y no se comprueban en runtime.

Herencia múltiple

Python permite heredar de varias clases. El MRO (Method Resolution Order) resuelve conflictos usando el algoritmo C3.

Métodos dunder

`__str__`, `__eq__`, `__len__`... permiten integrar objetos con el protocolo de Python.

Herencia y polimorfismo

Herencia en Python

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        return f"Hola, soy {self.nombre}"

class Docente(Persona):
    def __init__(self, nombre, dpto):
        super().__init__(nombre)
        self.dpto = dpto

    def saludar(self): # polimorfismo
        base = super().saludar()
        return f'{base}, del dpto. {self.dpto}'

class Investigador(Persona):
    pass # hereda todo sin cambios

# Herencia múltiple
class ProfesorInvestigador(Docente,
                           Investigador):
    pass
```

Conceptos destacados

Buenas Prácticas: PEP 8 y Docstrings

El **PEP 8** es la guía de estilo oficial de Python. Seguirla hace que el código sea más legible y mantenible. Los **docstrings** documentan módulos, clases y funciones de forma estándar, habilitando herramientas como `help()` y Sphinx.

Nomenclatura

- `snake_case` para variables y funciones
- `PascalCase` para clases
- `UPPER_CASE` para constantes
- `_privado` por convención

Formato

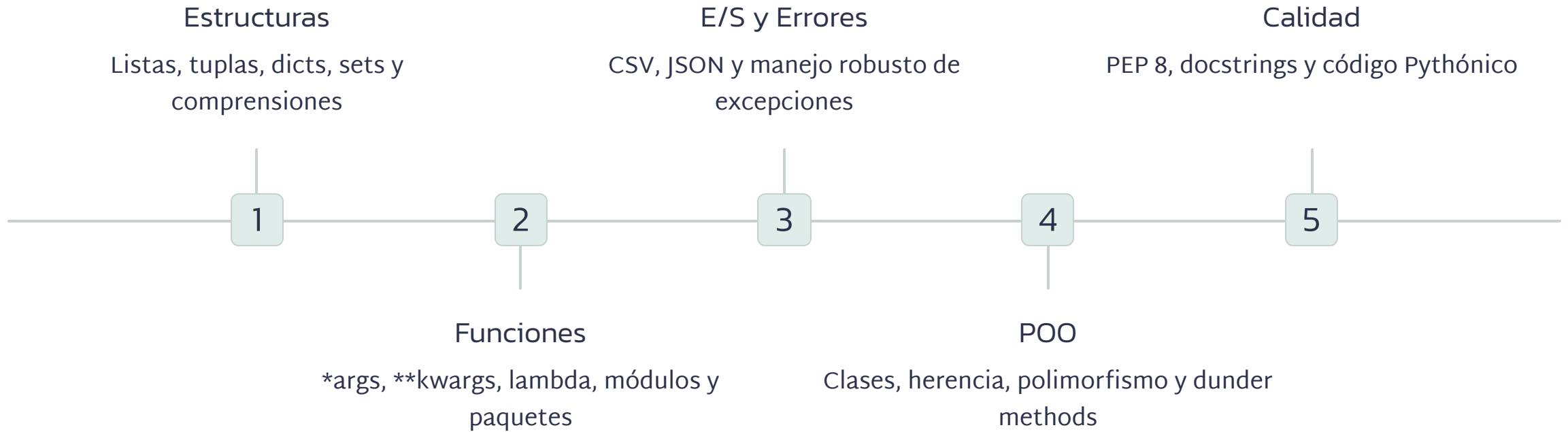
- Máximo 79 caracteres por línea
- 4 espacios de indentación (no tabs)
- 2 líneas en blanco entre funciones de nivel superior
- Espacios alrededor de operadores

Docstrings

```
def calcular_media(notas):  
    """  
    Calcula la media aritmética.  
  
    Args:  
    notas (list[float]): Lista  
    de calificaciones.  
  
    Returns:  
    float: Media de las notas.  
  
    Raises:  
    ValueError: Si la lista está  
    vacía.  
    """
```


Resumen y Próximos Pasos

Hemos recorrido los bloques constructivos esenciales de Python moderno. Ahora estás preparado para escribir código Pythonico, bien estructurado y mantenible.



❏ Practica combinando todos estos conceptos en un proyecto real: un gestor de alumnos en CSV/JSON con clases y manejo de excepciones es un ejercicio ideal.